

Lab 2 — Task 2: Manage Docker Containers and Images

Akash Singh Sagar | Repository: github.com/akashsinghsagar/devops-lab2

Objective

Manage the full container lifecycle (stop, start, remove), distribute an image through a self-hosted private registry, and orchestrate a two-service application with Docker Compose.

Step 1 — Container lifecycle: stop, start, remove

The lab2-app container from Task 1 is used to walk through the standard lifecycle commands: listing containers (including stopped ones), stopping, starting, and finally removing the container.

```
docker ps -a --filter name=lab2
docker stop lab2-app
docker ps -a --filter name=lab2
docker start lab2-app
docker ps --filter name=lab2
docker stop lab2-app
docker rm lab2-app
docker ps -a --filter name=lab2
```

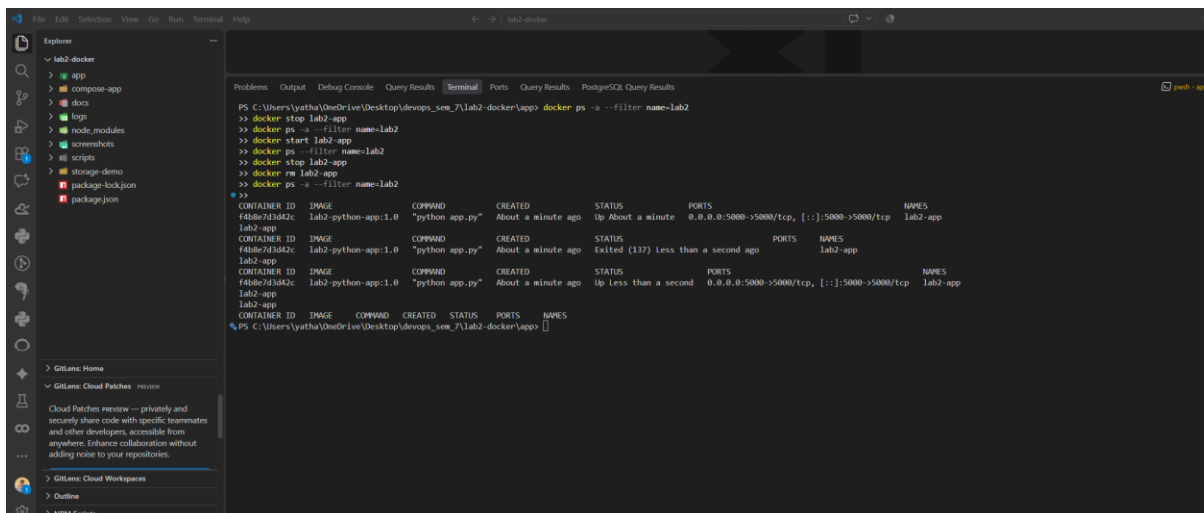


Figure 1 — The container's STATUS transitions from Up → Exited (137) → Up again after stop/start, then disappears from docker ps -a once removed.

Step 2 — Push the image to a local registry

A private Docker Registry (registry:2) is run locally on port 5050. The application image is re-tagged with the registry's host:port prefix, pushed, and the registry's HTTP API is queried to confirm the tag landed.

```
docker run -d -p 5050:5000 --name lab2-registry registry:2
docker tag lab2-python-app:1.0 localhost:5050/lab2-python-app:1.0
docker images lab2-python-app
docker push localhost:5050/lab2-python-app:1.0
curl.exe -s http://localhost:5050/v2/lab2-python-app/tags/list
```

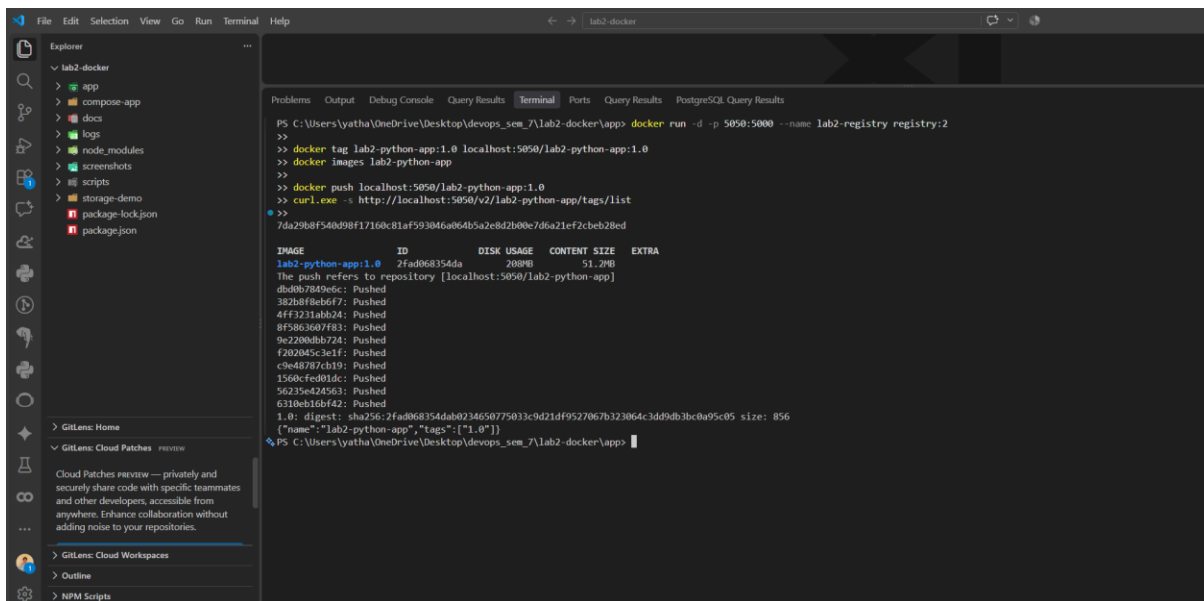


Figure 2 — All nine image layers report “Pushed”, and GET /v2/lab2-python-app/tags/list confirms tag “1.0” is now hosted on the local registry.

Step 3 — Docker Compose service definition

A two-service stack is defined with Docker Compose: web (built from ./web, exposing 5001→5000 and reading a REDIS_HOST environment variable) and redis (the official redis:7-alpine image, reachable only on the internal Compose network via expose, not published to the host).

```

services:
  web:
    build: ./web
    image: lab2-compose-web:1.0
    container_name: lab2-compose-web
    ports:
      - "5001:5000"
    environment:
      - REDIS_HOST=redis
    depends_on:
      - redis
  redis:
    image: redis:7-alpine
    container_name: lab2-compose-redis
    expose:
      - "6379"

```

Step 4 — Bring the stack up and tear it down

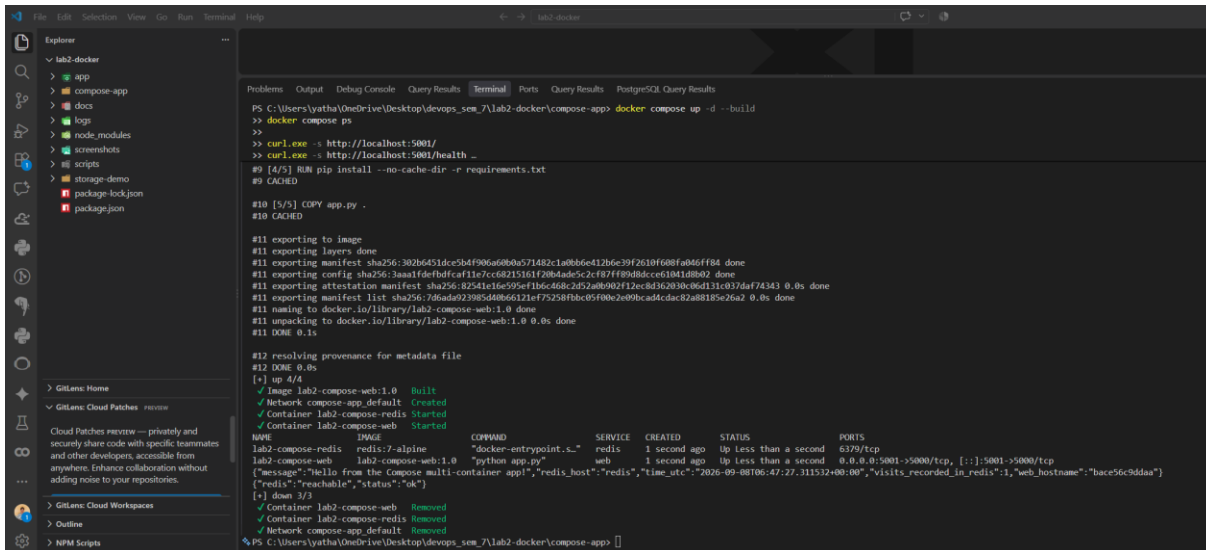
docker compose up -d --build builds the web image and starts both services; docker compose ps confirms they're running; the endpoints are exercised with curl; and docker compose down cleanly removes the containers and the Compose-managed network.

```

cd lab2-docker\compose-app
docker compose up -d --build
docker compose ps
curl.exe -s http://localhost:5001/

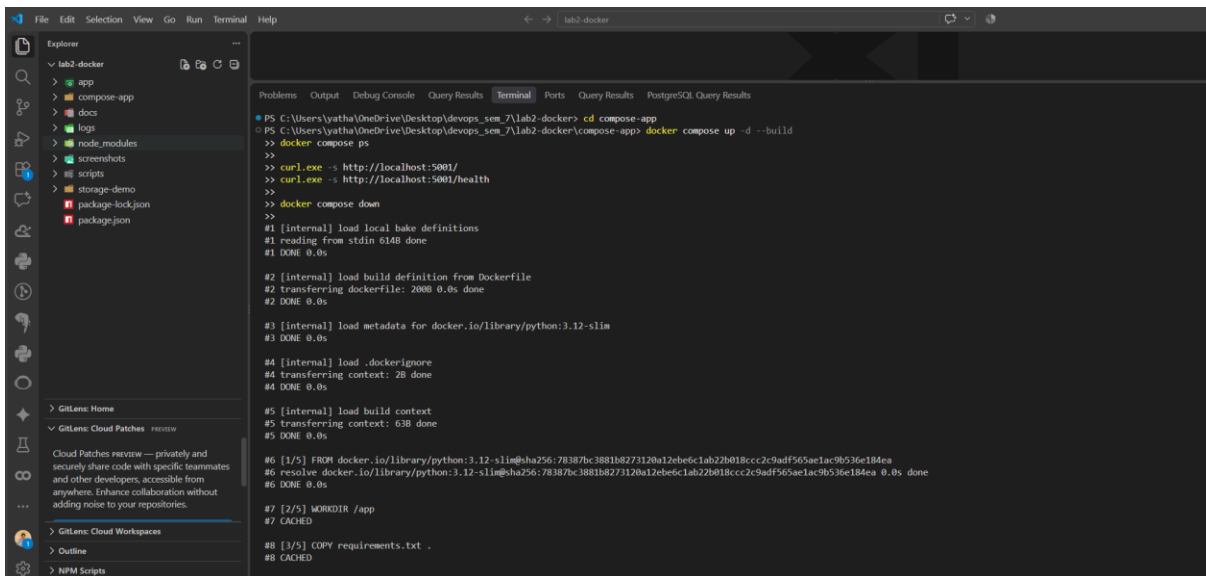
```

```
curl.exe -s http://localhost:5001/health
docker compose down
```



```
PS C:\Users\yatha\OneDrive\Desktop\devops_sem_7\lab2-docker> docker compose up -d --build
>> docker compose ps
>>
>> curl.exe -s http://localhost:5001/
>> curl.exe -s http://localhost:5001/health
>>
#0 [4/5] RUN pip install --no-cache-dir -r requirements.txt
#0 CACHED
#0 [5/5] COPY app.py .
#0 CACHED
#11 exporting to image
#11 exporting layers done
#11 exporting manifest sha256:302b6451dce5b4f906a6080571482c1a0bb6e412b6c39f2610f608fa06ff84 done
#11 exporting config sha256:3aa1fdefbdcf11e7cc68215161f20b4ade5c2c87ff89d8dce61041d8b02 done
#11 exporting attestation manifest sha256:82541e1e595ef1b6c468c2d52a8902f12ec8d362030c06d131c03daf74343 0.0s done
#11 exporting manifest list sha256:7d0ad922082d48b6d21ef72587b0c05f00c2e09bcad4dc02a0185e26a2 0.0s done
#11 naming to docker.io/library/lab2-compose-web:1.0 done
#11 unpacking to docker.io/library/lab2-compose-web:1.0 0.0s done
#11 DONE 0.1s
#12 resolving provenance for metadata file
#12 DONE 0.0s
[+] up 4/4
✔ Image lab2-compose-web:1.0 Built
✔ Network compose-app_default Created
✔ Container lab2-compose-redis Started
✔ Container lab2-compose-web Started
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS                PORTS
lab2-compose-redis  redis:7-alpine       "docker-entrypoint.s..." redis       1 second ago   Up less than a second 6379/tcp
lab2-compose-web    lab2-compose-web:1.0 "python app.py"        web        1 second ago   Up less than a second 0.0.0.0:5001->5000/tcp
{"messages": "Hello from the Compose multi-container app!", "redis_host": "redis", "time_utc": "2025-05-08T06:47:27.311532+00:00", "visits_recorded_in_redis": 1, "web_hostname": "bace56c9ddaa"}
[+] down 3/3
✔ Container lab2-compose-web Removed
✔ Container lab2-compose-redis Removed
✔ Network compose-app_default Removed
PS C:\Users\yatha\OneDrive\Desktop\devops_sem_7\lab2-docker> docker compose up -d --build
```

Figure 3 — Build output for lab2-compose-web:1.0 followed by both services reporting STATUS “Up”.



```
PS C:\Users\yatha\OneDrive\Desktop\devops_sem_7\lab2-docker> cd compose-app
PS C:\Users\yatha\OneDrive\Desktop\devops_sem_7\lab2-docker\compose-app> docker compose up -d --build
>> docker compose ps
>>
>> curl.exe -s http://localhost:5001/
>> curl.exe -s http://localhost:5001/health
>>
>> docker compose down
>>
#1 [internal] load local bake definitions
#1 reading from stdin 6148 done
#1 DONE 0.0s
#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 200B 0.0s done
#2 DONE 0.0s
#3 [internal] load metadata for docker.io/library/python:3.12-slim
#3 DONE 0.0s
#4 [internal] load dockerignore
#4 transferring context: 2B done
#4 DONE 0.0s
#5 [internal] load build context
#5 transferring context: 63B done
#5 DONE 0.0s
#6 [1/5] FROM docker.io/library/python:3.12-slim@sha256:78387bc38811b8273120412ebefc1ab22b018ccc2c9adf565ae1ac9b536e18dea
#6 resolve docker.io/library/python:3.12-slim@sha256:78387bc38811b8273120412ebefc1ab22b018ccc2c9adf565ae1ac9b536e18dea 0.0s done
#6 DONE 0.0s
#7 [2/5] WORKDIR /app
#7 CACHED
#8 [3/5] COPY requirements.txt .
#8 CACHED
```

Figure 4 — The web service’s JSON response shows “redis_host”: “reachable”, confirming inter-service DNS resolution; docker compose down removes both containers and the network cleanly.

Result

The full container lifecycle (stop → start → remove) behaved as expected, and the image was successfully distributed through a self-hosted registry at localhost:5050. Docker Compose brought up a two-service stack in which the web container reached redis purely by service name (REDIS_HOST=redis), demonstrating Compose’s built-in service discovery, and down removed every resource it created. Task 2 is complete.